Compiler Design (Midterm Lab 4)

Task: You are given an input file named **input.txt**. Each line contains **one statement**.

Your program must do the following:

Step A, Read and classify every line

For each line, classify it as one of these:

1. Variable Declaration
2. Function Declaration
3. Invalid

Write the classification for each line into **report.txt** in this format:
Line <n>: <classification>

Step B, Write only valid lines into a new file

Create **valid_code.txt** that contains only the lines that are classified as Variable Declaration or Function Declaration, in the same order.

Step C, Add a summary at the end of report.txt

At the end of report.txt, append:

- Total lines read
- Number of variable declarations
- Number of function declarations
- Number of invalid lines

Spaces may appear anywhere, but you can ignore spaces while checking.

1) Variable Declaration (only these forms)

Valid if it matches one of these patterns:

- int id;
- float id;
- double id;
- char id;

Where id must follow:

- first character is a letter or underscore
- remaining characters can be letters, digits, underscore

Examples:

- Valid: int x;, float _avg;, double a1;
- Invalid: int 2x;, int x y;, int x

2) Function Declaration (prototype, no parameters to keep it easy)

Valid if it matches:

- int fname();
- float fname();
- double fname();
- char fname();
- void fname();

Where fname follows the same identifier rule.

Examples:

- Valid: int sum();, void show();
- Invalid: int sum(int a);, int 3f();, int sum()

Sample input.txt

```
int x;
float avg;
int sum();
void show();
int 2x;
int f(int a);
```

Expected report.txt

```
Line 1: Variable Declaration
Line 2: Variable Declaration
Line 3: Function Declaration
Line 4: Function Declaration
Line 5: Invalid
Line 6: Invalid
```

Summary:
Total lines = 6
Variable Declarations = 2
Function Declarations = 2
Invalid = 2

Expected valid_code.txt

```
int x;
float avg;
int sum();
void show();
```